Privacy-Preserving and Trustworthy Cyber-Systems

CIS 4212/6214 (Spring 2026)

Instructor: **Dr. Attila A. Yavuz**
TA: ***Saif E. Nouma***

Homework 7 (Due: $May\ 1^{st}, 2026 - 23:59$)[100 Points + 15 Extra Credit]

(Upload your solutions to the CANVAS)
Requirements: HW groups are 2-3 people. Each student should contribute all parts of the answers. If one group member drops, the others are equally responsible (each HW can be handled by a single student if needed, so no excuse for missing a partner (if you are doing in group of 2), see syllabus for breaking-up with pair programming partner). Directly borrowing (e.g., copy-paste) from any material and putting in solutions (e.g, from online solutions, Wikipedia, or research papers) is plagiarism (see Syllabus for its corresponding actions). Please cite very carefully each resource you use but citing a solution does not give a license to directly put it as an answer. **All of your answers must be in your own words and interpretations. All your solutions will be passed through MOSS and Turnitin for plagiarism check.**
Note: Many of these questions require you to recall explanations/discussions during the class (this is why attendance is important), and also make research on the internet to find good answers. Please kindly note that very brief and/or informal explanations will potentially be not sufficient.
Remark: HW should be prepared by a text editor (e.g., Microsoft Word or Latex). Handwritten submissions are not accepted. In your answer key, please indicate the point of each question as it is done in this HW document. Preferably, take questions from here, plug them into your answer key with their sub scores, and provide your answer below them. This will facilitate the grading.
As indicated in Syllabus, **no partial credit is possible for coding answers that does not produce correct results with respect to test vectors, deviate from the description and rubric, are coded on platforms other than specified, or do not work on the grader's machine.**

0.1 Putting UMACs into Action - Aggregation Capabilities of Universal Message Authentication Codes with Applications [60 points]

In this assignment, you will put this knowledge into action by extending the concept with multi-block UMAC processing and aggregation. The UMAC with aggregations is used for various real-life applications such as network coding, intra-vehicular networks, and other similar wireless systems with resource limitations. This toy example will provide an experience in the theory and practice of such applied cryptographic notions.

1. Read the prime modulus q from the file **q.txt** (written in hexadecimal format). Note that q is a system-wide parameter of LC-UMAC.
2. Read the shared seed from the file **seed.txt** (written in hexadecimal format). The seed is exactly 32 bytes.
3. Read the message from the file **message.txt** and interpret it as a sequence $M = (m_1, m_2, \ldots, m_n)$, where each sub-message m_i is a $|q|$-bit block. The message size is *not fixed*, therefore your implementation *must* handle any bit size of both M and q. For simplicity, assume that $|q|$ always divides $|M|$ evenly, so that $n = \frac{|M|}{|q|}$. (note that $|x|$ denotes the number of bits in x).

4. Using the shared seed, invoke the ChaCha20 PRNG to generate a pseudo-random byte string of the required length. Specifically, derive $2n$ subkeys, each of bit-length $|q|$, arranged as n pairs $(a_1, b_1), (a_2, b_2), \ldots, (a_n, b_n)$. The byte string is parsed sequentially: the first $|q|$ bits form a_1, the next $|q|$ bits form b_1, the following $|q|$ bits form a_2, and so on. Store all a_i values in **a.txt** and all b_i values in **b.txt**, one value per line in hex format.

5. Compute the aggregate LC-UMAC tag by partitioning M into n blocks and signing each block m_i with its corresponding subkey pair (a_i, b_i), accumulating the individual MACs modulo q:

$$\sigma \leftarrow \sum_{i=1}^{n} \sigma_i \bmod q \quad \text{, where } \sigma_i = a_i \cdot m_i + b_i \bmod q$$

   where each a_i and b_i is a $|q|$-bit value derived from the PRNG output above.

6. Write the final aggregate LC-UMAC tag σ to the file **aggtag.txt** in hexadecimal format. You also need to write individual tags $\{\sigma_i\}_{i=1}^{n}$ in a separate file **tags.txt**, each per line in hexadecimal format as well.

Compile your code with the following command:
gcc lc_umac.c -lcrypto -o lc_umac
Run your code for the first test files with the following command and arguments:
./lc_umac q.txt seed.txt message.txt
Expected generated files:
a.txt, b.txt, aggtag.txt, tags.txt

NOTES:

- We have provided test files that you could use to test the correctness of your codes. In grading at our side, the seed, message, prime number (q), and lengths will be chosen at random and everything must work.
- Your code requires to read three input files: **q.txt**, **seed.txt**, and **message.txt**.
- You can test your implementation with the given bash script **VerifyYourLCMACSolutions.sh**.
- You must submit one file: **lc_umac.c**.

Rubric:

- **Correct a.txt and b.txt [15 pt]**
- **Correct tags.txt [15 pt]**
- **Correct aggtag.txt [30 pt]**

0.2 RSA and Multiplicative Homomorphism: Simplified Condensed-RSA [40 points]

Here, you will be implementing a simplified version of Condensed-RSA scheme that we have discussed using OpenSSL.

Let e and (d, n) are RSA public and private key pair, respectively, and $M = (M_1, \ldots, M_j)$ are j messages distinct messages to be signed. It is possible to compute an aggregated signature via condensed RSA signature as follows:

$$\sigma_{1,j} \leftarrow \prod_{i=1}^{j} H(M_i)^d \bmod n \quad \text{, where } H \text{ is the SHA-256 hash function.}$$

You need to implement this C-RSA signature using OpenSSL. Specifically, you will be given (e, d, n) and M, and you are asked to produce $\sigma_{1,j}$ as output. The implementation will be with OpenSSL.

STEPS:

1. Read from **rsa_params.txt** and convert RSA key pair (e, d, n) from hexadecimal strings to 2048-bit BIGNUM.
2. Read the message from the file **message.txt**, written in hexadecimal format.
3. Initialize $\sigma_{1,j}$ to 1.
4. Write a for loop to implement RSA multiplicative homomorphism.
   For each iteration $i \in \{1, \ldots, j\}$:
   (a) Hash the message of each block $h_i \leftarrow H(M_i)$, where $M_i \in M$ and M_i is of 128-byte size.
   (b) Convert hash value h_i from byte array (unsigned char) to BIGNUM instance
   (c) Generate individual signature: $\sigma_i \leftarrow H(M_i)^d \bmod n$
   (d) Accumulate the individual signature σ_i to the aggregate signature $\sigma_{1,j}$: $\sigma_{1,j} \leftarrow \sigma_{1,j} \cdot \sigma_i \bmod n$
5. Output the result of Multiplicative Homomorphism to a file **condensed_rsa.txt** in hexadecimal format. Moreover, store individual signatures $\{\sigma_i\}_{i=1}^{n}$ in a file **individual_rsa.txt** in hexadecimal format, each σ_i per line.

Compile your code with the following command:
gcc rsa.c -lcrypto -o rsa
Run your code for the first test files with the following command and arguments:
./rsa rsa_params.txt message.txt
Expected generated files:
condensed_rsa.txt, individual_rsa.txt

NOTES:

- This part uses 2048-bit RSA key pair.
- The file **message.txt** is given with variable byte size, but a multiplicative to 128. So, $M = (M_1, \ldots, M_n)$ where $\boldsymbol{n = \frac{m}{128}}$ (note that m is the total number of bytes in the given message.txt).
- In this HW, e and d are both 2048-bit integers. But, in practice, e must be less than $2^{16} - 1$ and $e < d$ for computation efficiency. Please be aware of that.
- You will need two input files to run the code:
  - **message.txt** - Contains the message to be signed
  - **rsa_params.txt** - Contains the RSA parameters (e, d, n), each is stored per line in hex format

Rubric:

- **Correct individual_rsa.txt [20 pt]**
- **Correct condensed_rsa.txt [20 pt]**

0.3 Extra Credit - Schnorr Signature Scheme: [15 points]

Given the signer's key pair (x, y) - where $y = g^x$ for g as the generator of Z_q^*, Schnorr signature generation (on message m) is as follow:
Select r at random from Z_q^*
Compute $R = g^r \bmod p$
Compute $d = H(m||R)$
Compute $s = r - x * d \bmod q$
Output $\sigma = (R, s)$

A. Explain the role of r in computing signature. More specifically, explain how r contributes to the security of the signature (discussed in the lectures in great details). **[5 pt]**

B. Sony PS3 used the ECDSA signature scheme. Explain the attack by focusing on what caused the attack and how the attack is executed mathematically. To gain points you must show the steps of the attack with precise mathematical formulations, and show how it can be launched to Schnorr signature described above. **[5 pt]**

C. Describe how Schnorr signatures are related to the Fiat-Shamir transformation. You must provide mathematical and algorithmic steps of Fiat-Shamir, and show how that is relevant to Schnorr digital signatures. **[5 pt]**

Write your solution to **hw7.pdf**